Kinematic Restitution Balancing (KRB)

A Low-Cost Analytical Fix for Energy Gain in Velocity-Level Impulse Solvers

Joshua Sideris
Gear3Games
https://gearbox2d.com
josh.sideris@gmail.com

Abstract

In discrete physics simulations using velocity-level impulse solvers (Sequential Impulse / Projected Gauss-Seidel), energy gain is a common numerical artifact. This paper introduces **Kinematic Restitution Balancing (KRB)**, a per-constraint correction for that leak in both unilateral constraints (collisions) and bilateral constraints (joints). By compensating for force-induced velocity drift (Component A) and auditing potential-energy shifts during position correction (Component B), KRB removes the two usual analytic energy sources inside a single-pass SI/PGS pipeline, without a second position pass. It does not claim a global energy invariant; evaluation and limitations sections state the coupled-constraint gap that follows from the per-constraint audit.

1. Introduction

Velocity-level sequential-impulse (SI) solvers [Catto 2005] are the standard real-time rigid-body approach: constraints are solved after explicit force integration with a fixed iteration count. KRB is a drop-in *preSolve* energy audit for that pipeline. It does not add solver passes, replace PGS, or switch to position-based dynamics.

Interactive rigid-body practice is surveyed by Bender, Erleben, and Trinkle [Bender et al. 2014]. Against that map: Baumgarte stabilization [Baumgarte 1972] and split impulse / nonlinear Gauss–Seidel (NGS) position passes [Catto 2014, 2024] hide immediate bounce-from-correction but do not tax the potential-energy work of the remaining displacement Δh; soft constraints, temporal Gauss–Seidel (TGS), and XPBD are alternative dampers [Catto 2011; NVIDIA Corporation 2024; Macklin et al. 2016]. Dual-pass and TGS schemes [NVIDIA Corporation 2024] trade extra cost per step for better convergence. Position Based Dynamics and XPBD [Müller

et al. 2007; Macklin et al. 2016] sidestep velocity-level restitution bias but are a different solver family. Geometric and variational integrators [Hairer et al. 2006] bound unconstrained Hamiltonian drift; KRB is not one of those. To our knowledge, prior velocity-level practice either accepts Baumgarte gain, hides bounce with a second position pass, or leaves the SI stack for PBD/XPBD—none apply an analytic PE/KE balance to the expected correction displacement Δh at setup for both contacts and joints inside a single-pass impulse solver.

Two analytic sources inject artificial mechanical energy during constraint resolution.

Force-integration drift. With symplectic Euler integration, velocity is updated before the constraint solve:

$$v_{\text{solver}} = v_t + a_{\text{ext}} \cdot \Delta t. \tag{1}$$

Restitution and joint bias see v_{solver} instead of the true relative velocity v_{impact}. For a perfect bounce ($e = 1$), the launch velocity gains $a_{\text{ext}} \cdot \Delta t$ extra speed per frame. For a joint at rest, the solver sees gravity-induced velocity and fights it every frame at the velocity level.

Potential-energy shift from position correction. Baumgarte stabilization [Baumgarte 1972] feeds constraint error back as a velocity bias $v_{\text{bias}} = \beta C/\Delta t$, displacing bodies by Δh to resolve penetration or joint drift. That displacement increases potential energy (mgh) without reducing kinetic energy. The added PE converts to KE on later frames, causing runaway energy growth.

Component B closes the leftover Δh audit gap inside a single-pass SI profile (see placement map above).

KRB applies an analytic PE/KE balance to the *expected* correction displacement Δh at constraint setup, for contacts and distance joints. The audit is per-constraint; coupled graphs can still exhibit offsets when one constraint forces work on another (see Section 6).

2. Method

KRB performs two independent corrections during constraint `preSolve`.

2.1. Component A: Force velocity compensation

With symplectic Euler, integration updates velocity before the constraint solve (Equation 1), so restitution and joint bias see v_{solver} rather than the pre-force relative velocity. Store the per-body force increment $v_{\text{force}} = a_{\text{ext}} \cdot \Delta t$ during integration and subtract it from the relative velocity seen at setup. That recovers the true impact

normal velocity before restitution or bias is applied:

$$v_{\text{impact}} = v_{\text{relative}} - (v_{\text{force},B} - v_{\text{force},A}). \tag{2}$$

Component A applies to all constraints whenever symplectic Euler integration is used, independent of the bias method. In the implementation listing (Listing 1), `relativeVn` is this corrected normal velocity after subtracting `forceVn`.

2.2. Component B: Kinematic energy balancing

Baumgarte position correction displaces bodies by an expected distance Δh along the constraint normal. External forces do work $W = m(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})\Delta h$ over that displacement. To keep mechanical energy consistent, the launch or bias speed must reflect that work: using $\Delta(\frac{1}{2}mv^2) = W$ gives a surface speed $v_{\text{surf}}^2 = v_{\text{impact}}^2 + 2(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})\Delta h$. When available kinetic energy cannot pay the tax, clamp with $\max(0, \cdot)$ and apply restitution:

$$v_{\text{surf}} = \sqrt{\max\big(0,\, v_{\text{impact}}^2 + 2(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})\Delta h\big)}, \tag{3}$$

$$v_{\text{final}} = e \cdot v_{\text{surf}} = e\sqrt{\max\big(0,\, v_{\text{impact}}^2 + 2(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})\Delta h\big)}. \tag{4}$$

For ground collisions ($\mathbf{a}_{\text{ext}} \cdot \mathbf{n} < 0$), Component B taxes launch velocity to pay for increased PE; for ceiling collisions ($\mathbf{a}_{\text{ext}} \cdot \mathbf{n} > 0$), it credits velocity for work against external forces.

2.3. Effective displacement prediction

Δh is the expected normal displacement used in the Component B work term—not the full penetration depth, but the portion the position solver will actually correct this step after accounting for launch motion and per-iteration caps. When velocity and position phases are decoupled, the kinematic bounce velocity v_{launch} partially resolves penetration d before the position solver runs. The remaining overlap after that motion is the effective depth

$$d_{\text{eff}} = \max\big(0,\, d - (v_{\text{launch}} \cdot \Delta t)\big). \tag{5}$$

Many engines clamp per-iteration position correction to $\Delta h_{\max}$. For example, Gearbox2D uses `MAX_POSITION_CORRECTION`. The audit must match the work actually performed:

$$\Delta h = \min(d_{\text{eff}}, \Delta h_{\max}) \cdot \Gamma, \tag{6}$$

where $\Gamma = 1 - (1 - \beta)^n$ is the cumulative Baumgarte fraction applied over n position iterations with factor β.

3. Constraint application

3.1. Contacts and speculative gaps

For physical overlap ($d > 0$), both Component A and Component B are required.

Speculative contacts are created before overlap ($d < 0$, a gap) to prevent tunneling. Because no position-correction work occurs yet, $\Delta h = 0$ and Component B is omitted; Component A remains critical so gravity-induced velocity is not treated as impact speed.

Apply restitution bias only when restitution is enabled and either the corrected normal velocity exceeds a small threshold, or a speculative gap is closing faster than the gap width allows. Resting contacts must not receive an $e = 1$ launch bias.

3.2. Distance joints and the zero-velocity paradox

Distance joints target zero relative velocity along the rod. Omitting Component B entirely leaks energy when Baumgarte stretch correction does work against gravity. Component B must be applied, but capped so a resting joint is not paralyzed.

Let $v_{\text{bias}} = \beta C/\Delta t$ be the ideal correction velocity for constraint error C. The audited correction is

$$v_{\text{bias\_actual}} = \sqrt{\max\bigl(0,\, v_{\text{bias}}^2 - 2(\mathbf{a}_{\text{ext}} \cdot \mathbf{n})(\beta C)\bigr)}. \tag{7}$$

Equation (7) is the same PE/KE audit with expected stretch displacement βC in place of contact Δh. If kinetic energy cannot pay the PE tax, the joint retains a microscopic Baumgarte sag—bounded audit rather than infinite stiffness at rest. This paper's setup recipe covers contacts and *distance* joints; other joint types are outside the listed implementation.

4. Implementation

KRB runs at contact and distance-joint `preSolve`, before the velocity iteration loop. Prerequisites: symplectic Euler with per-body `forceVelocity` ($a_{\text{ext}}\Delta t$), Baumgarte position correction with factor β, position iteration count n, and penetration slop.

4.1. Contact setup

```
// Component A: true impact normal velocity
forceVn = dot(bodyB.forceVelocity - bodyA.forceVelocity, n)
relativeVn = vn - forceVn

vBounce = -e * relativeVn
shouldBounce = enableRestitution &&
  (relativeVn < -RESTITUTION_THRESHOLD ||
```

```
   (depth < 0 && relativeVn < depth / dt))

if (shouldBounce) {
  if (depth > 0) {
    depthAfterVelocity = max(0, (depth - slop) - vBounce * dt)
    cumulativeFactor = 1 - pow(1 - beta, n)
    expectedDisplacement =
      min(depthAfterVelocity, MAX_POSITION_CORRECTION) *
      cumulativeFactor
  } else {
    expectedDisplacement = 0  // speculative:  Component A only
  }
  accVn = forceVn / dt
  workTerm = 2 * accVn * expectedDisplacement
  vSurfSq = relativeVn * relativeVn + workTerm
  vFinal = e * sqrt(max(0, vSurfSq))
  bias = (depth < 0) ? -max(vFinal, depth / dt) : -vFinal
} else if (depth < 0) {
  bias = -depth / dt
} else {
  bias = 0
}
```

Listing 1. Contact `preSolve` bias (Component A + B).

4.2. Distance-joint setup

```
float forceVn =
  (bodyB->forceVelocity - bodyA->forceVelocity).dot(normal);
float v_bias_ideal = beta * C / dt;
float expectedDisplacement = v_bias_ideal * dt;
float accVn = forceVn / dt;
float workTerm = 2.0f * accVn * expectedDisplacement;
float v_bias_sq = v_bias_ideal * v_bias_ideal;
float adjusted_v_bias_sq =
  max(0.0f, v_bias_sq - workTerm);
float v_bias_actual = sqrt(adjusted_v_bias_sq);
bias = (v_bias_ideal > 0 ? v_bias_actual : -v_bias_actual) - forceVn;
```

Listing 2. Distance-joint `preSolve` bias.

During the velocity solve, use the stored bias; Component A's $-\mathrm{forceVn}$ is folded into `bias` so `relative_vn` is not double-corrected each iteration.

5. Evaluation

Four datasets: Datasets A–B energy runs last $600\,\mathrm{s}$; Dataset C KRB-on continuation lasts $8\,\mathrm{h}$ ($t = 28800\,\mathrm{s}$); Dataset D reports native `world.step()` wall-time only (Figure 1: compile-time KRB on/off; Figure 2: whole-engine traces; traces in

`data/dataset-c/` and `data/timing/`). Not a patched-Box2D result.

Energy claims C01–C07 use $dt = 1/60$, $e = 1$, zero friction and damping, sleep disabled throughout. Components A and B are an analytic two-leak decomposition; the only empirical switch is compile-time full KRB versus `-DGEARBOX_DISABLE_KRB` (no isolated A-only or B-only traces). Mechanical energy is $E = E_k + E_p$; Dataset A includes rotational KE, Dataset B is translational only (the website adapters do not expose ω); on the overlapping Gearbox floor-bounce traces the two agree to about 10^{-6}. The reported ratio is E/E_0. Instantaneous samples (1 Hz for Datasets A–B; every 10 s for Dataset C) alias the bounce and are not the drift signal; 60 s windowed means appear in Figure 1b, Table 2, and Section 6; C07 uses 600 s windows on the 8 h enclosure series (hourly means in `data/README.md` differ). Table 1 summarizes horizons, measurement surfaces, and Dataset C outcomes.

Table 1. Evaluation protocol and Dataset C (KRB on, 8 h runs). C07 windowed E/E_0 is on 600 s windows of the 8 h trace, not a 600 s run. Dataset D is ms/step native timing, not a Gearbox-versus-Box2D CPU comparison.

Set	Scene	Horizon	Surface	Outcome
A	all	600 s	`log-energy`	Table 2
B	floor, cradle	600 s	WASM benchmarks	Table 3
C	enclosure	8 h	`log-energy`	in box; 600 s win. $1.009 \to 0.977$
C	cradle	8 h	`log-energy`	$[1.034, 1.093]$; mean 1.053
D	A + stack	timed steps	native `g++`	Table 4

Table 2. Dataset A ablation: E/E_0 at 600 s (compile-time KRB on/off).

Scene	KRB	E/E_0	Win. mean	Outcome
Floor bounce	on	0.999	1.000	bounded
Floor bounce	off	6.53	climbing	runaway
High-pressure circle	on	—	1.00	stayed in box
High-pressure circle	off	—	—	escaped, step 161
Newton's cradle	on	1.07	1.05 after $t = 300$	bounded offset
Newton's cradle	off	1.79	climbing	secular gain

Contact-only scenes match Section 1; Table 2 and Figure 1a–b summarize the floor bounce and high-pressure enclosure. With KRB, windowed means stay bounded; without KRB the floor runaway and enclosure escape match the SI leak at higher impact rate. Instantaneous 1 Hz samples of the on-curve alias the bounce (0.67–1.33); the windowed mean is the drift signal.

The cradle is better with KRB but is not an invariant (Section 6); Table 2 and Figure 1c show a bounded offset rather than the off-curve secular gain. Dataset C long-horizon checks are in Table 1.

Dataset B is whole-engine behavior: iteration counts, split impulse, restitution thresholds, and default damping all differ. It is not an ablation of KRB inside those engines. Traces are in `data/dataset-b-*-600s.csv`.

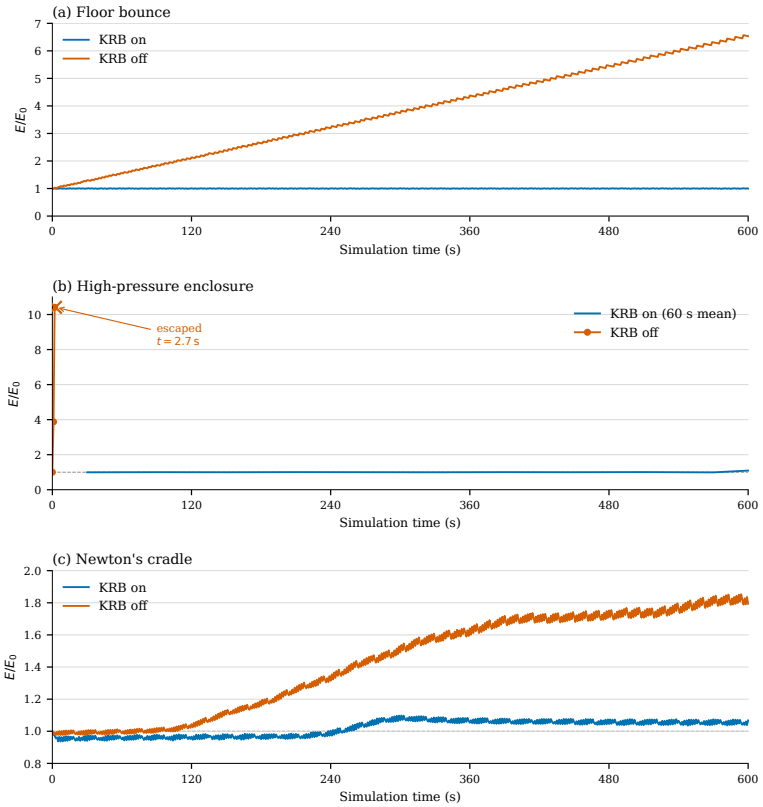

Figure 1. Mechanical energy ratio E/E_0 over $600\,\mathrm{s}$ of simulation time. (a) Single elastic floor contact. (b) High-rate enclosure; KRB on is a $60\,\mathrm{s}$ windowed mean, KRB off is raw $1\,\mathrm{Hz}$ samples terminating at tunneling ($t = 2.7\,\mathrm{s}$). (c) Newton's cradle (coupled contacts and joints). Vector original: `figures/fig-energy-ablation.pdf`.

Table 3. Dataset B: E/E_0 at $600\,\mathrm{s}$ on unmodified whole engines (not an in-engine KRB ablation; iteration counts, split impulse, and damping differ).

Scene	Engine	E/E_0	Outcome
Floor bounce	Gearbox2D (KRB on)	0.999	bounded
Floor bounce	Box2D-WASM	6.61	runaway
Floor bounce	p2.js	0.376	slow loss
Floor bounce	Matter.js	≈ 0	dead by $t \approx 160\,\mathrm{s}$
Newton's cradle	Gearbox2D (KRB on)	1.062	bounded offset
Newton's cradle	Box2D-WASM	0.221	stepped loss
Newton's cradle	p2.js	0.320	slow loss
Newton's cradle	Matter.js	≈ 0	dead by $t \approx 40\,\mathrm{s}$

Table 3 and Figure 2 compare whole-engine behavior (not an in-engine KRB ablation). Box2D-WASM floor bounce matches Gearbox KRB-off gain magnitude to a few percent;

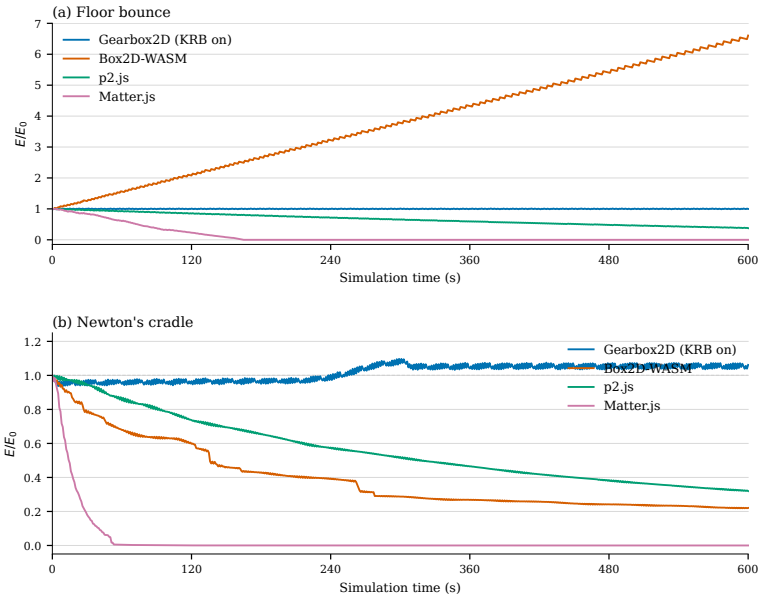

Figure 2. Mechanical energy ratio E/E_0 over $600\,\mathrm{s}$ on unmodified engines. (a) Elastic floor contact. (b) Newton's cradle. Gearbox2D is the default KRB-on WASM build; Box2D-WASM, p2.js, and Matter.js are unmodified whole-engine traces, not an in-engine KRB ablation. Vector original: `figures/fig-energy-engines.pdf`.

Gearbox KRB-on stays bounded as in Figure 1a. p2.js and Matter.js dissipate on the floor bounce.

On the cradle, unmodified engines lose energy rather than gain it (different failure mode from the contact leak). Gearbox KRB-on retains the bounded offset in Section 6, not the SI gain of Figure 2a.

This note reports a deployable technique with four datasets at eleven pages; JCGT shorts are typically near four pages, but this draft remains within the twelve-page venue ceiling.

5.1. Cost (Dataset D)

The "Low-Cost" claim is checkable without a second solver pass. Per dynamic body at integrate, KRB stores forceVelocity $= a_{\text{ext}} \cdot \Delta t$ (two floats). Per contact `preSolve` when KRB is on: a forceVn dot, optional Γ, a `workTerm`, one `sqrt`, and a bias update. Per distance-joint `preSolve`: forceVn, `workTerm`, one `sqrt`, with $-$forceVn folded into `bias`. There are zero extra PGS, velocity, or position iterations—not a second position pass or coupled-island PE tax. The paper recipe lists contacts and distance joints; whole-step timings below may include hinge Component A (engine-only).

Dataset D times `world.step()` on the same native `g++` binary with compile-time KRB on versus `-DGEARBOX_DISABLE_KRB` (`BENCH_FLAGS: -O3 -march=native -mfma -pthread -DGEARBOX_MT`). The product ships WASM; these numbers are a native ablation a reviewer recaptures via `make log-timing`. This is *not* a Gearbox-versus-Box2D CPU comparison (Dataset B is whole-engine energy, not in-engine ablation).

Scenes reuse Dataset A geometry for floor bounce, the high-pressure enclosure, and the cradle ($dt = 1/60$, $e = 1$, sleep off). Protocol: 100 warmup steps, then 1000 timed steps; three repeats; mean and range across repeats in ms/step (capture: i9-9900KF, g++ 13.3.0, `velocity_iterations`$= 50$, `position_iterations`$= 10$). We do not report a vanity percent speedup. Floor bounce and cradle on/off sit in the same scatter band; the enclosure shows a larger on/off spread (~ 0.012 versus $\sim 0.006\,\mathrm{ms/step}$) but both remain $\ll 0.02\,\mathrm{ms/step}$. A denser `large-stack` row (not Dataset A) is included for contact load; on/off also overlap ($\sim 0.38\,\mathrm{ms/step}$).

Table 4. Dataset D: native KRB on/off wall-time per `world.step()` (mean and repeat range, ms/step). Compile-time ablation with `BENCH_FLAGS`; not Box2D. Traces: `data/timing/timing-summary-{krb,nokrb}.csv`.

Scene	KRB on	KRB off	Notes
Floor bounce	0.010 (0.005–0.018)	0.009 (0.005–0.017)	Dataset A
High-pressure circle	0.012 (0.012–0.013)	0.006 (0.006–0.007)	Dataset A
Newton's cradle	0.048 (0.047–0.048)	0.049 (0.047–0.050)	Dataset A
Large stack	0.377 (0.372–0.381)	0.375 (0.369–0.380)	not Dataset A

6. Limitations

KRB is a per-constraint audit, not a coupled one. When resolving one constraint (for example a horizontal collision) forces a second constraint to do work against gravity (for example a pendulum rod lifting the body), the isolated PE/KE tax may miss the systemic change in potential energy.

Energy evaluation is restricted to the $e = 1$, zero-friction, zero-damping, sleep-disabled protocol in Section 5; it is not a general frictional or $e < 1$ claim. No experimental Component A-only or Component B-only ablation was run; full KRB on/off is the measured switch.

Newton's cradle is that coupled case. In Figure 1c and Figure 2b the KRB run shows a $\sim 7\%$ energy step near four minutes, then a plateau for the rest of the 600 s window—not a secular climb, and not a proof of a global invariant. Dataset C continues that KRB-on cradle to 8 h of simulation time ($dt = 1/60$, sampled every 10 s; traces in `data/dataset-c/`). After the same four-minute step, E/E_0 stays in $[1.034, 1.093]$ (mean 1.053) through $t = 28800$ s; the last-hour mean is 1.053, identical to the hour after the step, and 60 s windowed means do not jump again. That is an empirical multi-hour bound for this scene, still not a global invariant, and still not shown for arbitrary contact/joint graphs.

The same 600 s of unmodified Box2D, p2, and Matter (Figure 2b) dissipate rather than gain; that is a different failure mode and does not close the coupled-audit gap. Variational integrators [Hairer et al. 2006] remain a different claim.

A coupled-constraint audit is future work. High-speed tunneling, long ill-conditioned chains, and other scenes that fail for reasons other than the two leaks in Section 1 are outside the claim. The no-KRB enclosure escape in Figure 1b is the SI leak at high impact rate, not a KRB failure.

7. Conclusion

Kinematic Restitution Balancing is a drop-in correction for velocity-level sequential-impulse solvers [Catto 2005]. Component A removes force-integration drift from restitution and joint bias; Component B taxes (or credits) the launch or bias velocity for the potential-energy work of the expected correction displacement Δh. Both apply to contacts and distance joints, at the cost of a few extra scalars per constraint setup (Table 4).

That is a narrower claim than a dual-pass solver or a reduced-coordinate formulation. Split impulse and NGS [Catto 2014, 2024] still hide Baumgarte bounce by keeping position work off the physical velocity; KRB does not replace that machinery. It cancels the two analytic leaks in Section 1 inside a single-pass SI profile, with the coupled-constraint gap in Section 6 left open.

References

BAUMGARTE, J. Stabilization of constraints and integrals of motion in dynamical systems. *Computer Methods in Applied Mechanics and Engineering*, 1(1):1–16, 1972. URL: https://doi.org/10.1016/0045-7825(72)90018-7. 1, 2

BENDER, J., ERLEBEN, K., AND TRINKLE, J. Interactive simulation of rigid body dynamics in computer graphics. *Computer Graphics Forum*, 33(1):246–270, 2014. URL: https://doi.org/10.1111/cgf.12272. 1

CATTO, E. Iterative dynamics with temporal coherence. Presented at the Game Developers Conference, San Francisco, CA, 2005. URL: https://box2d.org/files/ErinCatto_IterativeDynamics_GDC2005.pdf. 1, 10

CATTO, E. Soft constraints. Presented at the Game Developers Conference, San Francisco, CA, 2011. URL: https://box2d.org/files/ErinCatto_SoftConstraints_GDC2011.pdf. 1

CATTO, E. Understanding constraints. Presented at the Game Developers Conference, San Francisco, CA, 2014. URL: https://box2d.org/files/ErinCatto_UnderstandingConstraints_GDC2014.pdf. 1, 10

CATTO, E. Solver2d. Box2D blog post, February 2024. URL: https://box2d.org/posts/2024/02/solver2d/. 1, 10

HAIRER, E., LUBICH, C., AND WANNER, G. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, volume 31 of *Springer Series in Computational Mathematics*. Springer-Verlag, Berlin, Germany, 2 edition, 2006. URL: https://doi.org/10.1007/3-540-30666-8. 2, 9

MACKLIN, M., MÜLLER, M., AND CHENTANEZ, N. XPBD: Position-based simulation of compliant constrained dynamics. In *Proc. 9th ACM SIGGRAPH Conference on Motion in Games (MIG)*, pages 49–54, Burlingame, CA, 2016. URL: https://doi.org/10.1145/2994258.2994272. 1, 2

MÜLLER, M., HEIDELBERGER, B., HENNIX, M., AND RATCLIFF, J. Position based dynamics. *Journal of Visual Communication and Image Representation*, 18(2):109–118, 2007. URL: https://doi.org/10.1016/j.jvcir.2007.01.005. 1

NVIDIA CORPORATION. Rigid body dynamics. PhysX SDK 5.4 Documentation, 2024. URL: https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/RigidBodyDynamics.html. 1

Index of Supplemental Materials

Paths are relative to `studies/kinematic_restitution_balancing/`.

- `jcgt/krb.tex`, `krb.bib`, `build-notes.md` — paper source.
- `KRB_Whitepaper.md` — number mirror (not authoritative).
- `figures/fig-energy-*.pdf` — Figures 1–2; `plot-energy-*.py`.
- `data/README.md`, `data/*-krb`, `nokrb.csv` — Dataset A.
- `data/dataset-b-*-600s.csv` — Dataset B; `data/dataset-c/` — Dataset C.
- `data/timing/timing-summary-krb`, `nokrb.csv` — Dataset D.
- `log-energy.cpp`, `log-timing.cpp` — loggers.

Author Contact and License

Joshua Sideris, Gear3Games — josh.sideris@gmail.com, gearbox2d.com. Supplemental code and data are ISC-licensed; see `LICENSE` in the Gearbox2D repository (github.com/JSideris/Gearbox2D).

Joshua Sideris, Kinematic Restitution Balancing: A Low-Cost Analytical Fix for Energy Gain in Velocity-Level Impulse Solvers, *Journal of Computer Graphics Techniques (JCGT)*, vol. ?, no. ?, 1–1, ????
????

Received:	August 22, 2026		
Recommended:	????-??-??	Corresponding Editor:	??? ???
Published:	????-??-??	Editor-in-Chief:	Alexander Wilkie

© ???? Joshua Sideris (the Authors).
The Authors provide this document (the Work) under the Creative Commons CC BY-ND 4.0 license available online at http://creativecommons.org/licenses/by-nd/4.0/. The Authors further grant permission for reuse of images and text from the first page of the Work, provided that the reuse is for the purpose of promoting and/or summarizing the Work in scholarly venues and that any reuse is accompanied by a scientific citation to the Work.